Plectis: What a Stranger Can Check

What public evidence can and cannot show about a private system

Will Cook

July 2026

Abstract

Plectis is a public software repository (an organised, copyable project folder) assembled from parts of a larger private system that cannot be published. It asks a narrow question: what may a stranger reasonably conclude from runnable fragments chosen by the person whose work is being examined? In the version studied here, each of the 88 registered components (separately testable parts) connects a claim to a fixed input, a command, saved output, a pass rule, and a stated limit. I call this a claim–evidence–limit contract. One component, for example, calculates an audio level from five small sample arrays. A reader can repeat the calculation and challenge the expected numbers. That run cannot show that the public code came from the private application or will work on other inputs. Across the collection, a public run does not by itself establish where the code came from, whether its answers are correct, whether the pass rule tests the claim as written, how the code behaves on untested inputs, or whether risks outside the declared scan are absent. I chose the published fragments, inputs, answers, rules, and limits. The repository therefore lets a reader test claims about its published files and outputs. It cannot establish the hidden system's history, whether the published selection resembles the private whole, or whether the private system works reliably. I report that I developed the private system by directing AI coding tools; the argument does not rely on that testimony.

1 The problem

I have a larger private software system that I cannot responsibly publish: it contains personal records, credentials, live browser and account information, my working notes, and unfinished tools. I would still like strangers to be able to test some claims about it instead of accepting my say-so. I use *answerable* in a deliberately local sense. A claim is answerable when a stranger can run a public procedure that bears on it, see how the result is judged, and point a disagreement at a named public input, rule, or output. This says nothing yet about whether the claim is true, important, or representative of the hidden system. This paper asks what conclusions that narrow form of answerability can support when the public fragments come from a system no reader can see.

Plectis is my working answer: a public software repository, stored at `github.com/wcook04/plectis`. A *repository* is an organised project folder, usually kept with its revision history; to *clone* a repository is to copy that folder and history to another computer. Everything this paper says a stranger can check is in that repository. This paper describes how its claims are exposed and where the evidence stops.

I am 22 and study economics. Over about ten months I built the private system by directing AI coding agents, software tools that can edit, run, and revise code across a task. I set objectives and review criteria, accepted or rejected their output, and sent them back to revise failures; the agents wrote nearly all of the code, and I wrote nearly none of it. No other person worked

on the system. This account explains why the publication problem arose and why the choices examined below are my responsibility, but it remains testimony about origin. Plectis can expose public files and runs; it cannot independently prove how the private work was produced. The public repository does not prove the story, and the argument does not rely on it.

The limit on what a public run can establish would be the same if every line had been written by hand. Authorship might affect how often a failure occurs: for example, an implementation and the program that checks it can share a mistake when one process drafts both. This paper compares no development methods and estimates no error rates.

The boundary of the evidence can be stated before any command is run:

Status	**What belongs in it**
Publicly checkable	The contents of a named public version, the commands listed for its components, the files those commands write, and the results of its public tests.
Reported here	The private system's history, the role of AI agents in producing it, and my claim that the published material came from the private system.
Not established here	Whether the public selection represents the private whole, whether the published code works beyond its fixed inputs, whether the project's own pass rules are correct or even test the stated claims, and whether the private system works reliably at all.

The paper examines one author-curated software collection. It defines the component contract, then follows one ordinary component from input to limit. Five distinctions separate what a passing run shows from conclusions it may appear to support. Two tables classify all 88 components by group and by the kind of public evidence they supply. The final sections identify the choices an evaluator would need to make independently of me.

This is a descriptive analysis of one public repository, not a statistical study. Runeson and Höst distinguish a case from the units of analysis within it [1]. I borrow only those two terms: the named repository version is the single case, and each component is a *unit of analysis* (one item examined inside that case). This paper does not otherwise claim a full case-study design. No subset of the components is used to estimate performance or represent the private system.

Reading this paper requires no programming. Reproducing its runs requires a terminal (the text window in which commands are typed), Git (the standard program for copying and versioning repositories), the Python programming language (version 3.11 or later), the `pip` installer (the standard tool for fetching and installing Python software), and basic command-line use. So that the body of the paper can stay with the argument, the exact commands, environment details, and installation notes are collected in the appendix, which is a dated record of one verification pass at the version named there.

2 The component contract

A *component* in Plectis is one registered, testable part of the project. Its *fixture* is a fixed sample input, and its *receipt* is a saved record of a prior run that the checker can read. A *validator* is the program that applies a component's pass and refusal rules. A *commit* is a named, saved version of the repository. These terms describe files and commands. Here *evidence* means the public input, procedure, output, and judgement rule offered in support of a claim. Calling them evidence does not establish their adequacy. *Contract* means only that the claim, those materials, and a limit are stated together so readers can check their fit. Nothing in these labels grades the

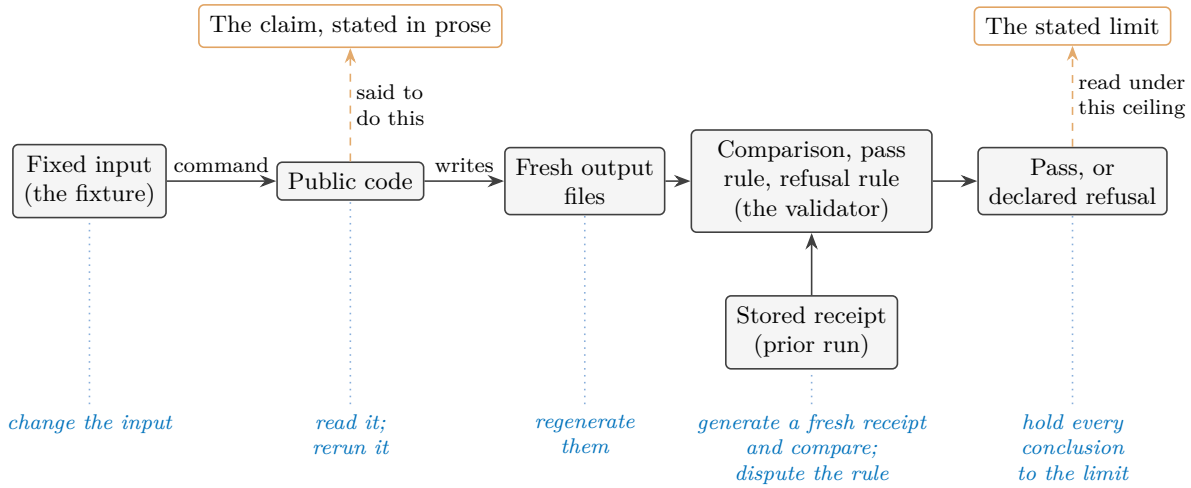

Figure 1: What every registered component must connect, at commit `57ffb7f5830c`. The top row is words; the middle rows are mechanism; a pass from the validator is offered as evidence for the claim and must be read under the limit. Every element, words and mechanism alike, is supplied by the project. Across the figures, a dashed outline marks something outside what the public run establishes. This figure has no dashed boxes: the claim and limit are public words, though they are not executable mechanisms. In Figures 2 and 3, dashes mark private material the reader cannot inspect or conclusions not established by the observed run. The italic blue row names what a stranger can do to each part without asking anyone. Amber, here on the claim, the limit, and the links asserting them, marks words and choices the project itself supplies; the same two colours keep these meanings in every figure. Section 3 runs one real component through exactly this chain.

importance of what a component does. (Inside the repository a component is called an *organ*, and the code package is named `microcosm_core`: the project was previously called Microcosm, and several file and command names keep the old name. The generated page `ORGANS.md`, named for the old term, gives a plain description and a first command for every component.)

At commit `57ffb7f5830c` the registry, the component list read by the programs, contains 88 entries. Every entry is required to connect the same things: a claim stated in prose; a fixture; a command; the output files the command writes; a stored receipt from a prior run; an explicit rule for what counts as a pass and what must be refused; and a stated limit on what a pass establishes. Figure 1 lays the required parts out, together with the fact about them that the rest of this paper keeps returning to: every part is supplied by me, and every part is exposed to a stranger.

An assertion about private software can only be believed or doubted as testimony. A component converts the assertion into a procedure, and a procedure can fail in front of a stranger. The conversion does not make the claim true. What it changes is the form disagreement can take: a reader who doubts a component's claim can point at a particular input, a particular line of the validator, or a particular number in the output, instead of arguing with my description of software they cannot see. A component that fails is still answerable in this sense, and a component can be answerable and trivial; answerability concerns the form a claim takes, not its truth and not its importance.

A component may also end in a declared refusal rather than an answer. The worked example below refuses an input format it does not support. Components that depend on an external tool, such as the Lean proof checker (a program that verifies mathematical proofs), must record a refusal with a stated limit when the tool is absent, rather than reporting success. *Runnable*

therefore means that the operation and its declared failure boundary are present in public form; it does not mean that every optional dependency exists on every machine. A refusal is informative only when it is declared in advance and distinguishable from a breakdown: a predeclared refusal marks the edge of what the component claims, while an unexpected crash is a defect. The two must not be read as the same kind of event. One honesty condition on refusals cannot be discharged from inside the repository at all: the public record shows each boundary as declared at the named commit, and it cannot show whether the boundary was drawn before or after an awkward case was first met. A boundary drawn afterwards turns a failure into a designed refusal in the telling. Saving the version before an outside evaluation begins, as Section 7 proposes, is what would make that ordering checkable.

The idea does not depend on software. The contract needs a claim in words; a determinate public case; a procedure a stranger can cause to run; an observable result; a rule connecting result to claim; a declared boundary of application; and a stated ceiling on inference. That the case here is a file of numbers, the procedure a command, and the record a text file is a convenience of this example. The same contract could be satisfied in another field by a laboratory protocol with declared rejection conditions, or by a sealed evaluation service with published scoring rules. I make no claim about those settings; the point is only that the contract is a shape for exposing claims to strangers, and this repository is one example. Read simply as software, it is a test suite with a registry and unusually complete paperwork. That is a fair description. The paper asks what such a test suite lets a stranger challenge, and which conclusions still lie beyond it.

This local notion overlaps with reproducibility and transparency but is not a synonym for either. A run can repeat perfectly with no stated claim attached, and a claim can be answerable while still awaiting a good test. The design also begins from partial disclosure: it offers one limited point of contact, not a view of the hidden system. A component exposes the published fragment to challenge. It does not make the private whole answerable.

Answerability is not the same property as computational reproducibility, although the component contract supplies materials needed to test the latter. Under the convention used by the National Academies, computational reproducibility means obtaining consistent results using the same input data, computational steps, methods, code, and conditions of analysis; such consistency does not by itself establish correctness [2]. Software testing calls the mechanism used to decide whether an output is correct an oracle, and the difficulty of obtaining a trustworthy oracle is a recognised testing problem [3]. The risk that a curated collection disproportionately exposes favourable cases is analogous to selective publication and the file-drawer problem [4]. An *assurance case* is a structured argument linking a system claim to reasons and evidence. The Object Management Group's SACM 2.3 standard models such cases as auditable claims, arguments, and evidence; its annex maps the Claims–Arguments–Evidence notation onto SACM [5]. Plectis is not an implementation of that standard and makes no claim that the private system is safe. Its validator applies a published rule; it does not supply the full argument from evidence to claim that an assurance case would require. ACM calls digital research materials such as code, scripts, and data *artifacts*. Its *Artifacts Evaluated—Functional* badge means those materials passed an independent audit for documentation, consistency, completeness, ability to run, and evidence of verification and validation. *Results Reproduced* means another team later obtained the main results using some author-supplied artifacts [6]. Plectis claims neither status. These standards are comparison points, not statuses awarded here. This paper claims no new theory of assurance, testing, or reproducibility; it reads each public procedure against its claim and stated limit.

3 One run, examined

The component `batch8_audio_level_rms_port` computes a loudness level from short arrays of audio samples. I chose it for the worked example because it is ordinary. It needs no mathematics beyond a square root and no knowledge of the private system, and the claim it makes is small. Its value here is that every part of the evidence can be seen at once.

The reported origin is this. The private system contains a small macOS recording application, and one of that application's functions reduces a short stretch of audio samples held in memory to a level between zero and one. The public component re-expresses that calculation in Python. The component's code and its stored receipts name the private file it claims to re-express (`AudioLevelMonitor.swift`); a stranger cannot open that file, so the name is a report, not evidence. What the stranger can check is everything else.

The declared calculation begins with root-mean-square (the `rms` in the component's name): square each sample, average the squares, and take the square root. It then applies the project's gain of eight and caps the result at one. Signed sixteen-bit integer samples are first divided by 32,767, the largest positive value in that representation, to bring them onto the same scale. An empty input must produce zero. An unrecognised format name must be refused rather than guessed at.

The component's public command runs it against its fixture and writes fresh output outside the repository (the backslashes split one long command across lines):

```
PYTHONPATH=src python3 -m \
  microcosm_core.organs.batch8_audio_level_rms_port run \
  --input fixtures/first_wave/batch8_audio_level_rms_port/input \
  --out /tmp/plectis-audio-rms \
  --acceptance-out /tmp/plectis-audio-rms-acceptance.json
```

At commit `57ffb7f5830c`, the run produced:

Case	Input	Expected	Observed	Result
Decimal samples	0.0, 0.05, −0.05, 0.1	0.489897949	0.489897949	pass
Sixteen-bit integer samples	0, 1638, −1638, 3277	0.489892970	0.489892970	pass
Values above the cap	1.0, −1.0, 0.5	1.000000000	1.000000000	pass
Empty input	(none)	0	0	pass
Unknown format	`pcm24`	refusal	refusal	pass

Because the inputs and the rule are both printed above, the first row can be checked without the repository or a computer: the squares of the four samples average to 0.00375, whose square root is 0.061237…, and eight times that is 0.489897…. A reader with a calculator can recompute this expected value from the stated formula alone. For this one case the reader can act as their own *oracle*, the general name for a tester or mechanism used to decide whether an output is correct. Here elementary arithmetic supplies a standard independent of the program under test.

Almost nothing else in the collection is like that, which is why the example matters. Elsewhere the project usually supplies the implementation, the input, the expected values, and the validator through the same author-controlled process. A test assembled that way can pass for four different reasons: the implementation and the expectation are both correct; the two share a mistake; the chosen cases happen to sit where the implementation behaves; or the validator checks a weaker property than the prose claim suggests. A matching result therefore establishes

repeatability under that public test. Independent evidence bearing on correctness needs a credible oracle whose expected result is justified independently of the implementation, as elementary arithmetic is for the first row. Fresh inputs alone do not supply one.

The example suggests a question for any software test: where did the expected value come from, and who supplied it? The answer changes what a match can support:

Where the oracle lives	**What a matching result then supports**
The project itself: implementation, cases, expectations, and rule from one author-controlled process.	Repeatability under the project's own test; the four readings above all stay open.
The reader's own derivation from stated premises, as in the worked example's decimal-samples row.	Correctness of that one calculation on that one case.
An established external tool or reference.	Whatever the reference itself warrants, on the cases run, within the reference's own limits.
An evaluator who chooses inputs and independently justifies expected results after the version is saved.	Independent evidence bearing on correctness for those cases, within the oracle's own justification and limits.
The world: an outcome observed apart from any program.	Evidence about a real-world outcome, where one exists.

Most components in the collection sit in the first position, except where a reader promotes a case to the second by recomputing it, as the worked example's decimal-samples row invites. Components that invoke a named external tool occupy a hybrid of the first and third positions: the project still chooses the case and the rule, while the tool supplies a result that inherits its own limits. The evaluations in Section 7 describe ways to move consequential choices out of the project's control.

The refusal row records a designed boundary, not a failure: an input naming an unsupported encoding (`pcm24`, a format this component does not handle) must be rejected with an error, and is. The stored reference for the whole table is `batch8_audio_level_rms_port_validation_receipt.json` under `receipts/first_wave/batch8_audio_level_rms_port/`, so a fresh run is compared against a fixed record rather than against memory. A stored record by itself does not prove that the stated command produced it; its use is that anyone can generate a fresh one and compare the two.

The limit is part of the component. This run checks one public Python calculation over synthetic sample arrays (arrays composed for the test, not captured from a device). It does not run the original application, start an audio session, request microphone permission, or capture sound, and it cannot establish that the named private file is where the calculation came from. Nor does the table show that the task was difficult (it was not), that the component is useful elsewhere, or that the remaining entries resemble it. What it shows is that one assertion now has a form in which a stranger can rerun it, dispute its rule, perturb its input, or replace my expected value with a better-derived one.

The calculator check supplies an independent expected number for one row. It does not establish that the component came from the private system, show that the validator covers the full prose claim, predict untested inputs or unselected components, or rule out publication risks beyond the project's declared scans. Even its evidence for correctness is limited to that one calculation. The next section treats these five gaps separately.

4 Five distinctions

Figure 3 sets out five ways a matching run can tempt a reader to conclude too much. Each subsection separates what a reader can observe from what it cannot establish, then names the evidence needed to go further. Component limits are specific instances of these gaps.

Public execution versus where the code came from

A reader can establish what a named public commit contains and what its commands do. No public run establishes where the material came from. Plectis records that boundary. Each copied file has a claimed source and a SHA-256 fingerprint: a fixed-length value calculated from its contents and used to check for change. Two different files can in principle produce the same value. SHA-256 is designed to make finding such a pair infeasible in practice, not impossible [7]. A match therefore supports a narrow claim: the public file's contents agree with the version represented in its public record. But I control both sides, so that agreement is evidence of internal consistency, not of origin. The same holds for the audio component's named source file: the name is an entry I wrote. Origin claims could only be strengthened by something a public repository cannot supply about itself, such as commitments published before the fact, or a trusted person permitted to examine both sides. The term *provenance* means an object's origin and history. Here, that origin remains reported rather than publicly established.

Repeatability versus correctness

A run that reproduces the stored receipt shows that the public code, on the supplied input, in a sufficiently similar environment, produces the same result again. Correctness is a different property: it needs a standard for what the result should have been, and a standard is only worth something if it does not merely restate the implementation. The worked example makes the difference concrete. Its first row has an oracle independent of the project, namely elementary arithmetic; most components have only the project's own expected values and validator. Passing tests anywhere in Plectis should be read at that strength and no more.

A validator's rule versus the stated claim

A pass is a verdict from the validator, and a validator checks exactly the property written into it, which can be narrower than the claim as worded. A component whose prose speaks of handling a class of inputs may in fact verify that one output file exists and matches a stored shape; a rule can hold while the sentence it stands under goes untested. The two previous distinctions do not cover this gap: a result can be repeatable, and correct as far as the checked property goes, while the checked property is beside the stated point. Plectis makes the gap inspectable rather than closed. Every validator is public, so a reader can put the rule beside the prose and object where the rule is weaker, and the review in Section 8 makes exactly that comparison its central step. Since the same author-controlled project produced both the claims and the rules, agreement between them records one judgement twice, and no count of passing components can substitute for reading one rule against one sentence.

Selected cases versus general behaviour

Selection acts at two levels. Within a component, the fixture covers only a few possible inputs. Across the collection, the published components are my selection from a private whole.

I chose which mechanisms could be given public form, which inputs to include, which expected values to store, and which pass rules to enforce. Nothing in the public record shows how many candidates there were, what was excluded beyond the privacy rules, or whether the collection resembles ordinary work in the private system. A collection can consist entirely of genuine items and still give an unrepresentative picture of what it was drawn from. This is analogous to selective publication and the file-drawer problem, not the same process [4]. Figure 2 draws the situation: public checking reaches everything to the right of the boundary, and the pool the selection was drawn from stays on the left.

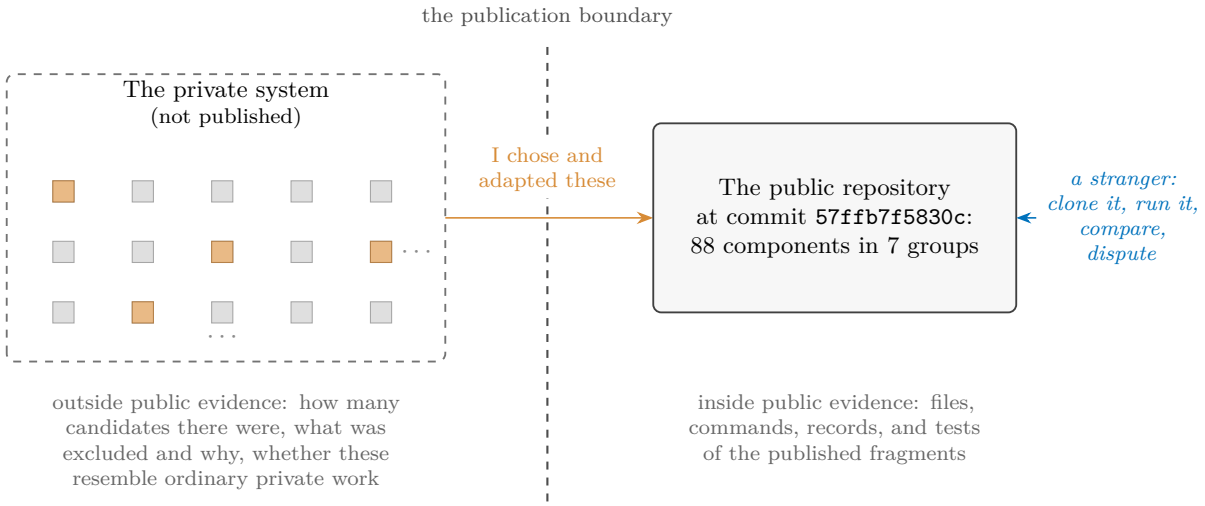

Figure 2: The selection problem. Each square stands for a private mechanism; the darker ones were given public form. Each public component has the parts shown in Figure 1. The grid trails off because its true extent is exactly what the public side cannot know. The count of squares on the left, the rule that picked the darker ones, and the resemblance between the two sides are all invisible from the right. The collection supplies directly checkable evidence about the published files and their behaviour under named tests, but no basis by itself for generalising to the hidden whole.

The registry's count should be read in that light. A registry of 88 entries defines exactly what is published. A later evaluation would need that list to draw a sample and report failure rates against a fixed total. The count is an inventory, not a score, and even the unit of counting is a choice I control, since one mechanism could have been registered as several components, or several as one.

Risk reduction versus guarantee

The publication process runs checks whose honest description is that they reduce specific risks. Automated scans search copied material for specified patterns of credentials, personal information, and private file paths, and record what was omitted or rejected; a scan establishes that the scanner did not find the patterns it was designed to find, and nothing stronger. Pages such as the component map are generated from the registry, and tests compare page and registry, which prevents the particular failure of prose drifting away from records; a generator and its records can still be wrong together. The documented `tour` run takes the public checkout as its explicit input and does not automatically discover a neighbouring private repository. The release-export contamination check is an explicit exception: when a copy of the private project folder is available, it may compare the contents of public source files with private files to detect leakage. None of this amounts to a guarantee that no secret, no identifying detail, and no

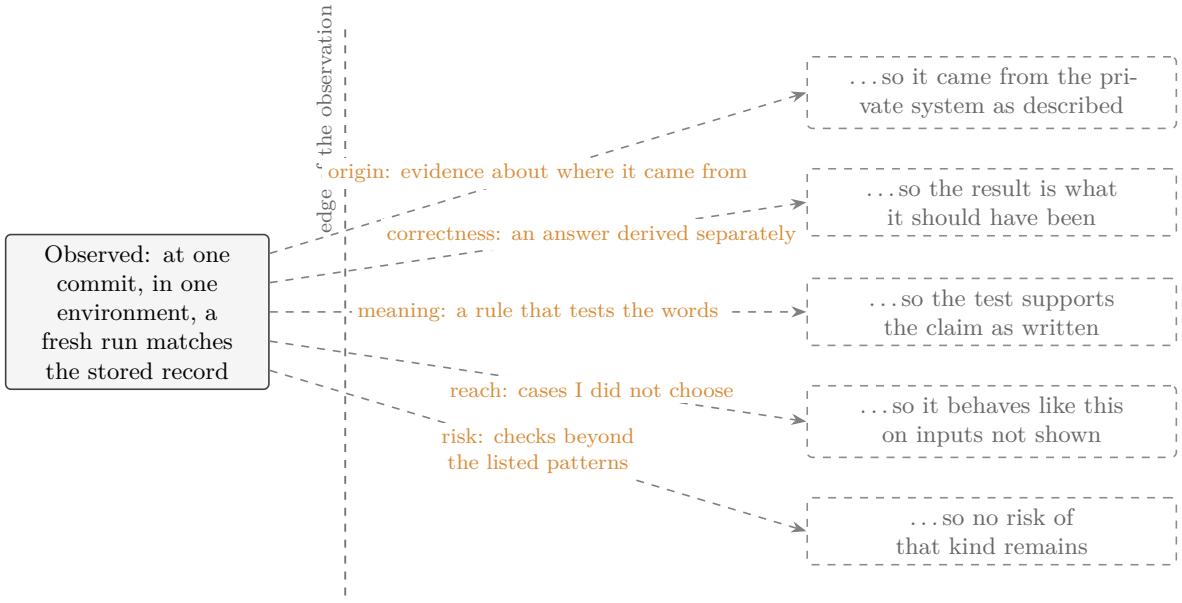

Figure 3: Five conclusions a matching run invites and does not license. Each dashed arrow needs a further assumption that the observation itself cannot supply. From top to bottom, the tags follow the order discussed above: origin (provenance), correctness, meaning, reach, and risk. The contract's contribution is not to cross these gaps but to make them nameable, one component at a time.

inconsistency survives. Publication remains a judgement about acceptable risk, and I made it.

Origin evidence and privacy pull in opposite directions. Establishing provenance would require someone to examine private material, and the privacy constraint that motivates the whole design forbids exactly that examination in public. Some claims are therefore permanently in the reported category, short of a confidential review. The honest treatment is to label them, not to dress them up as public proof through bookkeeping that one maintainer controls.

The same mistake lies behind all five: treating an observation as if it proved something never tested. A broader conclusion needs an added assumption. For origin, one must assume that the public object came from the private one. For correctness, one must know that the expected answer was right; for meaning, that the rule tested what the claim said. For reach, one must assume that the chosen cases stand for the unchosen; for risk, that the scan covered the relevant risks. The observation itself supplies none of these assumptions. The contract makes each one explicit. Once named, a gap can be disputed, assigned a cost to address, or deliberately left open. An unnamed gap is easy to cross without noticing. These five are not exhaustive. Two more gaps appeared earlier: observing a refusal today does not show when it was drawn (Section 2), and observing a number at one commit does not describe the next (the appendix). These are simply the five gaps this collection most needs a reader to hold.

5 What the collection contains

At commit `57ffb7f5830c` the 88 components fall into 7 groups:

Group	Count	What its components do
Getting started	2	Show a new reader where to begin and guide a first inspection.
Project mapping	12	Map files and written rules, find relevant material, and route a question to the part of the project that owns it.
Computer-checked mathematics	20	Check records and fixed examples from formal mathematics, including small Lean statements and one exact finite calculation.
AI reliability and safety tests	20	Replay fixed cases involving hostile instructions, software boundaries, stored information, tool use, and benchmark rules.
Research-method tests	9	Exercise fixed cases involving replication, conflicting forecasts, explaining model behaviour, simulation, and laboratory safety.
Public-copy maintenance	20	Check copied source, generated pages, publication boundaries, and disagreement between records and derived files.
Work recovery	5	Record unfinished changes and test how work resumes after interruption.

The groups do not share a subject; computer-checked mathematics has nothing in particular to do with audio levels or with recovering interrupted work. Their unity is the contract of Section 2: these are the places in the private system where a mechanism could be turned into a small, runnable public procedure. The collection is organised by how its claims are checked, not by subject.

The registry also classifies what kind of evidence each component supplies. It calls this classification a *route*. A route says whether the public component preserves source text, re-creates behaviour on fixed cases, records a run, or checks a declared rule. The four routes below cover all 88 entries. Each leaves a different question open:

Route	Count	What it preserves	What it cannot show
Record-checked copied source	21	The exact text of non-secret source files, checked against a public import record.	That the recorded origin is real; record and file share one maintainer.
Public reimplementation	27	One calculation's behaviour on fixed cases.	That it came from the private system; the private text is absent.
Direct or external-tool run	22	A local computation, or a named external tool's result, captured with its context.	Behaviour where the tool or environment differs; the result is as strong as the tool and the case.
Contract checker	18	Whether a public input satisfies a declared structural rule.	Whether the declared rule is the right rule to require.

The copied-source route preserves the most text and demonstrates the least behaviour. A reimplementation demonstrates behaviour while surrendering textual identity; the worked example is one of the 27 in that route. An external-tool run inherits whatever confidence the tool deserves, along with the tool's limits. A contract checker is worth exactly the contract its author chose to require. A reader who wants to know what kind of evidence a given component offers can read its registry entry, which names its command, its fixture, its receipt paths, its route, and its limit; the plain description of what it does lives on the generated component page named in Section 2.

Two cautions govern any reading of these tables. The first caution is arithmetic: components are not independent witnesses. Entries share code, fixture style, validator authorship, and my judgement; several entries can descend from one private mechanism, and where they do, their verdicts tend to move together. 88 entries are 88 registered claims, not 88 separate confirmations of anything. The same discount applies across the routes: they impose four different limits on what may be concluded; they are not four independent methods confirming one claim. Pages, registry entries, and receipts generated from the same underlying records repeat one witness rather than supply several. Machine checks can catch accidental disagreement among those records. They cannot create an independent witness.

The second caution is association. Group names such as computer-checked mathematics and AI reliability and safety tests borrow gravity from serious fields, and the borrowing happens in a reader whether or not any sentence asserts it. The corrective is to restate each group as the weakest property its validators actually check, which is often that a fixed case replays or that a record matches a declared shape, and then see how much of the impression survives. The names are for finding components, not for weighing them.

6 The author's hand

All five distinctions return to control. At the commit this paper describes, I hold every consequential choice on both sides of every test. I chose which private mechanisms were candidates, which were selected, and how finely each was divided into components, and so what the count counts. I chose which of the four routes each took, which inputs went into the fixtures, what the expected values are, what each validator checks, and where each refusal boundary sits. And I chose which group each entry is filed under, which failures this paper explains and in what tone, which single component the worked example honours, and where the publication boundary lies. Somebody had to make each of these choices, and the privacy constraint meant it was me. None of that is misconduct.

But that concentration of choice permits a failure mode, and it involves no false statement anywhere. Select the mechanisms that show well. Divide the favourable ones finely. Narrow each claim until its fixed cases satisfy it. Give awkward material the least demanding check. Declare as refusals whatever would otherwise fail. File small replays under weighty names. Then let the counts, the categories, and the machinery of records accumulate into an impression of scale and maturity that no individual sentence asserts. A document can be cautious sentence by sentence while its arrangement still overstates, because a reader's impression forms on what is counted, named, worked through, and left out, as much as on what is claimed. This paper is inside its own arrangement, so declaring the risk does not discharge it; nothing in the text can establish that the text is not doing what this paragraph describes.

Making disagreement local can also displace the larger question. A reader can settle an argument about one input or one rule while leaving untouched whether the selection is fair, whether the whole is sound, or what any of it is for. An objection about selection is not answered by the successful execution of any number of selected items. The registry hosts the small arguments; it does not settle the large ones.

Two corrections need no cooperation from me. First, read any component at the weakest property its validator checks, as Section 5 proposes. Second, treat every count as an inventory, and every agreement among my records as one witness. A third point concerns what no reader can inspect: a private mechanism is easiest to publish when it can be separated into a small task, gives the same output from the same input, and runs in isolation. A collection like this therefore over-represents pure calculation, replay, and record-checking, and under-represents

stateful, entangled, long-running behaviour. The published picture tilts towards what publishes well before any question of honesty arises.

Disclosure alone cannot remove this concentration of control. NASA's standard separates technical independence (evaluators did not develop the system), managerial independence (they choose what and how to assess), and financial independence (funding is controlled outside the development organisation) [8]. Plectis has had no such evaluation. Within the five gaps examined here, the narrower principle is that evidence for a claim becomes less dependent on me as the relevant choices stop being mine. The principle is local to those five gaps. Section 7 lists independent evaluations that address the dependencies this paper has identified.

7 What stronger evidence would look like

A passing run shows that named public code produced a stated result from a supplied input in a stated environment. It does not establish private provenance, representativeness, correctness beyond self-supplied criteria, usefulness, security, originality, or the private system's reliability. The computer-checked mathematics label covers different checks: proof-work records, small public statements checked by Lean, and one exact finite calculation. The label itself supplies no theorem; each entry supports only its own public claim and stated limit. At the named commit, all evidence still came from the project itself. No outside evaluator had independently rerun the repository or controlled any selection, expected answer, comparison, or provenance review.

Five forms of independent evaluation could add evidence that the public repository cannot supply by itself. These are not the four publication routes in Section 5. A publication route classifies evidence already in the registry; the evaluations below describe new work by someone outside the project. The order is only for exposition. It ranks nothing, and no evaluation is a prerequisite for another. Figure 4 shows which choice each evaluation transfers and which claim it still leaves open. The common question is who makes the choice that matters for that claim.

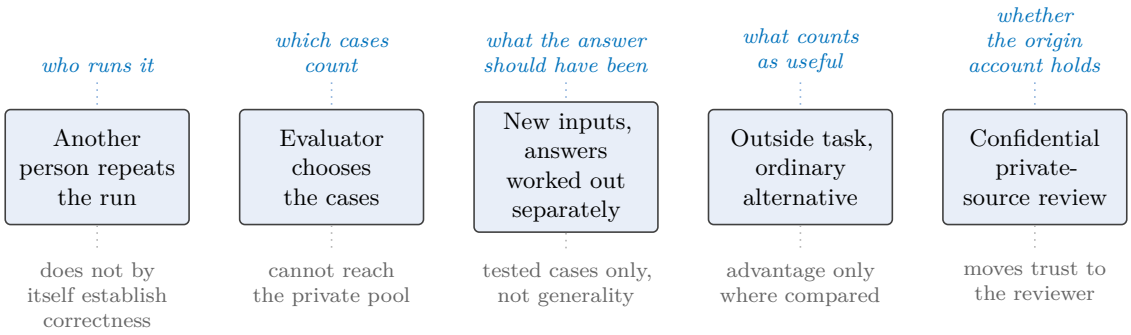

Figure 4: Five forms of independent evaluation. Above each evaluation, in italic blue, the choice that leaves my hands; below it, in grey, what that evaluation still cannot show. The evaluations are not cumulative stages. At the commit this paper describes, none has happened.

Independent repetition means a person with no involvement in the project reruns the supplied commands in their own environment and publishes what happened, including failures. That would reduce dependence on my machine and test whether the published instructions suffice elsewhere. It would not by itself establish correctness.

Evaluator-controlled selection means the evaluator chooses components from the full registry, without substitution when one proves awkward, and original failures stay on the record even if I later repair them. The units themselves are open to the same challenge: an evaluator who finds

that two entries are one dependent mechanism, or that a boundary excludes the difficult part of a task, is disputing the registry, not misusing it. That addresses curation within the public collection. It cannot address whether the collection represents the private system, because the private pool stays invisible.

Fresh inputs with independently derived expectations can move repeatability towards evidence bearing on correctness: inputs chosen after the version is saved, and expected results worked out by a method that does not pass through my code, whether independent calculation, a second implementation, or an external reference. The result would support claims about those cases only to the extent that the derivation is credible and independent and the cases are adequate; general correctness still would not follow.

Comparison on outside tasks is what a claim of advantage over ordinary alternatives would need: someone bringing a task that was not designed around the project's own categories and comparing the component against an ordinary alternative, which for several components would be a short conventional script. Nothing in this paper makes that comparison.

A provenance review, among the five evaluations shown here, bears most directly on the origin story: a trusted person examining selected private material against the public collection under confidentiality. Prior public commitments could provide another kind of provenance evidence. Even completed, this review would transfer trust rather than remove it: public readers would be relying on the reviewer's access, competence, and report instead of on mine, and the review itself could not be published. The privacy cost may reasonably never be paid. If it is not, the origin story remains testimony, and this paper is written so that its argument does not depend on it.

How the record should age

The evaluations above may expose failures. Repairs should add facts rather than erase them. The appendix therefore keeps three failures at the pinned commit even though a later commit repairs one. That repair is a fact about the later version; it does not subtract the first fact about this one. If outside criticism shows a validator too weak or a claim too broad, the revision should be legible as a revision, carrying its date and its cause, rather than the project appearing always to have meant the narrower thing. A test rewritten after its outcome is known is a different test and may be better. The known case can show whether the repaired code later breaks in the same way; fresh confirmation requires new cases whose answers were not used to design the repair. Disappointment is in scope: an evaluation that concludes a component is trivial, or no better than a short ordinary script, is the machinery working, and its conclusion belongs on the record with the same standing as a pass. An outside evaluator's report, where one comes to exist, should remain a separate document under the evaluator's control, cited rather than absorbed. Where the repository's future conduct falls short of this paragraph, the shortfall is itself a finding, and Section 8 says where findings go.

These evaluations do not pair one-for-one with the distinctions. They address the runner, public-case selection, tested-case correctness, usefulness, and origin. None repairs a validator–claim mismatch, proves representativeness, or turns risk reduction into a guarantee.

8 Conclusion

Plectis does not make the private system answerable as a whole. It puts particular public claims within reach of challenge by publishing a fixed case, procedure, output, rule, and limit. A pass is no stronger than its rule or than the basis for the expected answer.

Adding more author-controlled components enlarges the inventory; it does not alter who chose the cases or rules. Moving a relevant choice out of the author's hands can strengthen evidence about that choice; it does not repair every gap.

A first review. Use the appendix's no-install block to copy the repository and select the version analysed here. The tour describes the checkout but does not choose a component claim. Ask the repository to find the audio example with

```
PYTHONPATH=src python3 -m plectis comprehend --first-action "audio level calculation" --format text
```

For another claim, replace the quoted words. The response names a component, command, receipt, limit, and `no-write variant`: stored reference receipts stay untouched, while fresh files go under `.microcosm/first_action_runs/`. Run it, compare the receipts, read the validator's rule, and change an input where practical. Ask whether the rule supports the prose and any refusal boundary; report discrepancies through `CONTRIBUTING.md`. A pass licenses only the published rule. Section 7 lists broader evidence, most of it absent.

A A dated reproduction record

Unless a paragraph gives a later date, everything in this appendix is an observation about one commit, made on 17 July 2026 in one environment: macOS 15.6.1 on Apple silicon (arm64) with Python 3.13.7. The commit's full identifier is

`57ffb7f5830cc446a2cbd8083d5d80bcab2af563`

abbreviated `57ffb7f5830c` elsewhere. Later commits and other machines may give different results; the body does not depend on those differences.

Names. The repository is `github.com/wcook04/plectis`. As Section 2 notes, the code package inside it is named `microcosm_core`. The installed command is `plectis`, and an older command name, `microcosm`, still works as an alias.

Getting the code and running without installing.

```
git clone https://github.com/wcook04/plectis
cd plectis
git checkout 57ffb7f5830c
PYTHONPATH=src python3 -m plectis tour --format text .
```

The final full stop in the tour command means "the current folder". The `git checkout` line pins the clone to the exact state this appendix describes; omit it to run against the newest version instead, and expect the dated numbers below to differ if you do.

In a clean clone at this commit, the tour read 5,018 project files, found five reading orders, chose the README-first order (internally `readme_onboarding_route`), wrote 21 generated files under `.microcosm/`, and left the examined source files unchanged. Each observation names its evidence, event, and limit, and the generated directory can be rebuilt. The README gives a different file count from another day; both counts describe dated runs, not stable project size.

Installing.

```
python3 -m pip install .
plectis tour --format text .
```

The program needs no extra Python library; `pip` may fetch `setuptools` to build it. No check enforces the absence of network or model calls, so inspect the source.

The worked example. The command in Section 3 writes under `/tmp/plectis-audio-rms`, overwriting it on a rerun. A fresh run reproduced every displayed pass and value. One stale field reports a copied private module; the saved import record reports none because public code replaced it. The calculation is unchanged, but the discrepancy remains.

A second Python version. On 18 July 2026, a clean macOS clone used Python 3.12.7. Offline, the install used the already-present `setuptools` 80.9.0 (the build requires version 77.0.3 or later). The tour and example matched without another runtime library. This tests 3.12.7 and 3.13.7, not every supported version.

Project-wide checks. At this commit, `make check` loads the component registry in under a second. The Lean trust checker searches the source text of 58 Lean files for unfinished proof placeholders; custom axioms (assumptions added without proof); native calculations relying on more than Lean's proof-checking kernel; `partial` definitions, which Lean treats as opaque in its logic; `unsafe` definitions, which Lean excludes from mathematical reasoning; and removed computation limits. This search cannot show that each formal statement says what its author intended [9]. The registry and trust checks passed; the 58 files are not the 20 mathematics components. At this commit, the limited `make test` selection (not every test file) ran 361 tests from 32 public test files: 356 passed, two were skipped, and three failed. The failures were a line-break expectation in `AGENTS.md`; outdated stored line or byte counts in copied-source manifests; and root-layout rules that did not yet allow the `paper` directory or root PDF. A later change repairs the last failure without altering this pinned run. Exact test names, environment, and counts are in `paper/public-test-receipt.json`. `make ci` would add smoke and package-install checks, but its larger floor was not green because `make test` failed. These checks concern only the public checkout; this is my run, not an independent repetition.

The paper itself. `scripts/check_public_system_paper.py` fails if counts, pinned facts, the example, cautions, or citations disagree. Historical facts come from the pinned commit; current prose is checked separately. With one maintainer, this is internal consistency, not external truth.

References

[1] Per Runeson and Martin Höst. "Guidelines for Conducting and Reporting Case Study Research in Software Engineering." *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.
[2] National Academies of Sciences, Engineering, and Medicine. *Reproducibility and Replicability in Science*. Washington, DC: The National Academies Press, 2019. doi:10.17226/25303.
[3] Elaine J. Weyuker. "On Testing Non-Testable Programs." *The Computer Journal*, 25(4):465–470, 1982. doi:10.1093/comjnl/25.4.465.
[4] Robert Rosenthal. "The File Drawer Problem and Tolerance for Null Results." *Psychological Bulletin*, 86(3):638–641, 1979. doi:10.1037/0033-2909.86.3.638.
[5] Object Management Group. *Structured Assurance Case Metamodel (SACM)*, version 2.3, OMG formal/23-05-08, October 2023.
[6] Association for Computing Machinery. "Artifact Review and Badging—Current," version 1.1. Accessed 18 July 2026.
[7] National Institute of Standards and Technology. *Secure Hash Standard*, FIPS PUB 180-4, 2015. NIST FIPS 180-4.
[8] NASA. *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, Section 4.4.1.2, 2022. NASA-STD-8739.8B.
[9] Lean Project. *Lean Language Reference* sections "Validating a Lean Proof" and "Partial and Unsafe Definitions." Accessed 18 July 2026.